Rheinisch-Westfälische Technische Hochschule Aachen
Alessandro Berti

Phase Review: 3rd Sprint of the Implementation of "Conformance Predictions"

Group 4:
Richard Schuppener, Cem Colak

Contents

1 Summary of Sprint 3 **3**

2 Codebase Structure **3**

3 Testing **3**

4 Requirements Coverage Status - Overview per Module **3**
4.1 IO 3
4.2 Feature Extraction 4
4.3 Model Training 5
4.4 Evaluation 6
4.5 Config & Pipeline 7
4.6 Project Infrastructure & Testing 8

5 User Guide **9**

6 Team settings and Workflow **9**
6.1 Team settings 9

7 Sprint review **10**
7.1 Richard Schuppener 10
7.2 Cem Colak 10

1 Summary of Sprint 3

In this sprint, we focused on implementing all the remaining functionalities required for a fully functional end-to-end pipeline for conformance predictions. We intend to use the final sprint to further refine and optimise these functionalities, so that we can deliver a well-aligned and polished software artifact that fulfils Alessandro's requirements in the most sophisticated way. We also tested the functionality of the whole artifact for all options and combinations of options, as we can finally say that the artifact is functioning as intended and appropriate for a third sprint.

mention the last achievements

2 Codebase Structure

The codebase remained organized as in spint 2, so no further final changes.

3 Testing

Our testing modules are also complete and covering all the expectations stated in the project profile document.

4 Requirements Coverage Status - Overview per Module

4.1 IO

All Covered

- **LL-01.1**: `io.discover_process_model()` supports Inductive, Alpha, and Genetic Miner via PM4Py, enabling automatic model discovery when no normative model is available.
- **LL-01.2**: `visualization.visualize_process_model()` renders the model as Petri net, BPMN, or Process Tree, with optional file saving or direct display.
- **HL-05**: Event log loading, model import, and model discovery are all functional; the foundational data-acquisition step of the pipeline is complete.

- **LL-05.1**: `pm4py.read_xes` for XES logs; `pd.read_csv` combined with `pm4py.format_dataframe` for CSV logs in `io.load_event_log()`.
- **LL-05.2** (partially): Petri net loading via `pm4py.read_pnml` implemented; BPMN files loaded via `pm4py.read_bpmn` and Process Tree files via `pm4py.read_ptml`, both internally converted to a Petri net.
- **LL-05.3**: Inductive, Alpha, and Genetic Miner available as fallback discovery algorithms when no normative model file is provided.
- **HL-04**: `try/except` blocks in `main.py` catch and report setup errors, preprocessing failures, and pipeline exceptions to `stderr` with descriptive labels.
- **HL-01**: End-to-end prediction of conformance deviations is now fully operational; the *Future Deviations* target is wired through the complete pipeline – label extraction via `real_deviation_cost`, configurable threshold T, classification training, and evaluation.
- **LL-05.2**: Standalone loading of BPMN and Process Tree files as direct model inputs (without forced internal conversion to a Petri net).

4.2 Feature Extraction

All Covered

- **HL-06** (partially): `feature_extraction.compute_alignments()` invokes `pm4py.conformance_diagnostics_alignments`; log-moves, model-moves, and synchronous moves are extracted per case. Distance-to-completion indicators are not yet included.
- **LL-06.1**: PM4Py alignment dictionaries are computed for every trace against the normative Petri net using the standard cost function. Alignments are now keyed by case ID and re-emitted in first-appearance order, eliminating the silent ordering issue that previously misaligned diagnostics with cases.
- **LL-06.2**: Alignment moves are parsed by `real_deviation_cost()` to count genuine deviations (log-moves and visible model-moves) per case; cumulative alignment cost is extracted while silent (τ) routing moves are correctly excluded.

- **HL-07**: `feature_extraction.extract_features()` now supports control-flow features (Bag-of-Activities), alignment-based diagnostic features (Bag-of-move-types per activity), case-attribute features, and the hybrid combination of all three.
- **LL-07.3**: Control-flow frequency matrix constructed via `pd.crosstab` over case IDs and activity names, producing a per-case Bag-of-Activities feature matrix.
- **HL-13**: Alignment results are computed once per run and stored in memory; they are passed as a variable to avoid redundant re-computation within a single pipeline execution.
- **HL-02**: Structured case attributes are fully integrated as predictive features. `list_case_attributes` and `summarize_case_attributes` detect both explicitly prefixed (`case:`) and constant-within-case attributes (XES-style), and the CLI lets the user select a subset of them.
- **LL-07.1**: `generate_prefixes()` expands each case into varying-length prefix pseudo-cases, with optional outcome-leakage gating, supporting both *exactly-k* and *varying* $1..k$ modes.
- **LL-07.2**: `extract_remaining_time()` and `extract_temporal_features()` together produce a numerical feature matrix tailored to the remaining-time regression target, including elapsed time, time since the previous event, and event count.
- **HL-06**: *We left this out.* Addition of distance-to-completion indicators (e.g. remaining enabled transitions, token distance to the final marking).
- **LL-13.1**: *We left this out.* Hash-based caching of prefix activity sequences to deterministically identify and skip redundant alignment computations.
- **LL-13.2**: *We left this out.* Disk-based persistence of alignment results via `joblib`.

4.3 Model Training

All Covered

- **HL-08**: `train_classification_model()` wraps a configurable classifier (Decision Tree, Logistic Regression, or Gradient Boosting) in a `sklearn.Pipeline`. The binary deviation label is derived from the case's real alignment cost via the configurable threshold T, so the target is fully driven by alignment-based deviation cost.
- **LL-08.1**: `StandardScaler` is integrated as the first step in every sklearn Pipeline, ensuring scale-consistent feature distributions.
- **HL-09**: Baseline (control-flow features only) and conformance-aware (diagnostics or hybrid features) model variants are both supported via the `PredictionFeature` configuration.
- **LL-08.2**: Additional ML algorithms are available beyond the Decision Tree: *Logistic Regression* and *Gradient Boosting* for classification, and *Random Forest*, *Gradient Boosting*, and *Decision Tree Regressor* for regression. All are exposed via the CLI and selectable per run.

4.4 Evaluation

Covered

- **HL-16**: `evaluation.compute_classification_metrics()` computes accuracy, precision, recall, F1-score, ROC-AUC, and Brier score for binary and multi-class classification targets.
- **HL-12**: Confusion matrix heatmap and per-class accuracy bar chart are generated via Matplotlib in `visualization.save_evaluation_visuals()`.
- **HL-14**: The system allows for precise measuring of runtime and memory usage.
- **HL-14.1**: Each critical and potentially time-consuming step (often the pipelines) is precisely measured and the runtime is appended to running list. This functionality is performed by `measure_runtime`, which utilizes `time.perf_coutner` internally.
- **HL-14.2**: The peak memory consumed by the program is tracked by the function `measure_memory`. Additionally, it accounts for OS dependent units (bytes or kilobytes).
- **HL-14.3**: A counter accumulates the runtime from different steps in out program: from heavy *PM4Py* methods for discovery, test-train split etc., to whole pipelines that extract features or train models.

- **HL-14.4**: The generated performance data (runtime, peak memory usage) is printed to the CLI and can optionally be saved to a JSON. The runtime can also be visualized as a pie-chart.
- **HL-15**: The main pipeline uses `pm4py.split_train_test` with a user-configurable train ratio (default 0.7).
- **LL-16.2**: `compute_mae`, `compute_mse`, and `compute_rmse` are available in the evaluation module for evaluating remaining-time regression targets.
- **HL-17**: `evaluation.metrics_by_prefix_length()` slices the predictions of a single pooled model by prefix length and reports the full metric set per length; the CLI prints this table whenever prefix lengths vary.
- **HL-16**: *left out.* Conformance quality metrics for the normative model (fitness), and wiring of the regression metrics into the pipeline's `compute_regression_metri` hook so they are reported automatically for the remaining-time target.

4.5 Config & Pipeline

Fully Covered

- **HL-03**: `main.py` provides an interactive CLI that guides the user through model path/discovery selection, prediction target (future deviations, process outcome, remaining time), feature basis (including case attributes), prefix mode, train/test split ratio, and ML algorithm configuration.
- **LL-03.1**: Prediction target and feature basis options are fully defined in `config.py` and selectable in the CLI; both the case-attribute feature path and the conformance-deviation target are now fully wired through the pipeline.
- **HL-10**: `main.py` combined with `pipeline.py` forms an end-to-end runner from raw data and model input to evaluation output, without manual intermediate steps.
- **HL-14**: Runtime and resource measurement now wrap conformance checking, feature extraction, model training, and evaluation; the totals are reported at the end of every run.

- **LL-14.1**: `evaluation.measure_runtime()` uses `time.perf_counter()` around alignment computation, feature extraction, model training, and evaluation.
- **LL-14.2**: `evaluation.measure_memory()` polls `psutil.Process().memory_info()` to track peak RSS during the entire pipeline run.
- **LL-14.3**: Sub-second precision wall-clock measurements are recorded per step (conformance checking, feature generation, training, evaluation).
- **LL-14.4**: Runtime and resource metrics are printed to the console and can be persisted to a structured JSON log file at the end of the run.

4.6 Project Infrastructure & Testing

Covered

- **HL-18**: PM4Py, scikit-learn, pandas, NumPy, and Matplotlib are used as the primary library stack throughout all modules.
- **HL-19 & LL-19.1-19.2**: The project is organized into `io`, `feature_extraction`, `model_training`, `evaluation`, `pipeline`, `visualization`, and a centralized `config` module; Git is used for version control. Note: the conformance logic is consolidated into `feature_extraction` rather than placed in a dedicated `conformance` module. A formal branching strategy per module was not adopted, as we are working with one branch per team member.
- **HL-20**: `requirements.txt` and `README.md` are present in the repository.
- **LL-20.1**: `requirements.txt` as a plain text file is implemented and considered satisfactory for dependency management.
- **HL-11**: An automated pytest test suite covering IO, feature extraction, model training, and evaluation is executable via a single command (`pytest tests/`) and runs in the GitLab CI pipeline on every push or merge.
- **LL-11.3**: `test_generate_prefixes_lengths_and_content` asserts that a prefix of length k contains exactly the first k events of the original trace; the related `test_pooled_extraction_broadcasts_labels_without_leakage` test additionally confirms that outcome activities are gated out of the feature vector.

- **LL-11.1**: Dedicated pytest fixtures providing a synthetic, fully conforming XES log and a matching Petri net (currently the tests use ad-hoc DataFrame helpers).
- **LL-11.2**: Explicit test assertion that the alignment cost equals 0.0 on a fully conforming synthetic dataset.
- **HL-21**: Technical documentation of system architecture.

5 User Guide

The User Guide is now fully implemented and mature to use as a reference for users who want to understand the options within the usage of the CLI. We believe that the User Guide is a crucial part of our Software Artifact, as it allows users with few to no background in process mining to make valuable use of our software.

6 Team settings and Workflow

As already stated in the first sprint review, we rotated responsibilities for this sprint. Cem was responsible for IO, Model Training, and Feature Extraction, while Richard was responsible for the new visualization module, evaluation, and pipeline. Both of us were jointly responsible for the CLI and config modules, as they are relatively small and we wanted to work on them together to maintain a shared understanding of the user interface and configuration options. This division of responsibilities worked well, as it allowed each of us to understand and reflect on the other's past contributions and to extend them with new capabilities.

For the next sprint, we plan to keep the same split on the implementation side and place a stronger focus on documentation. We intend to work together on the User Guide, the technical documentation, the README, and the phase review. This will allow us to review each other's content under the four-eyes principle, ensuring more robust output quality.

6.1 Team settings

Throughout the three sprints, we maintained a consistent rotation of responsibilities across all modules, although we kept the per module split more stable for the last two sprints since the third sprint was more about adding the last functionalities and focusing on documentation and finalization for

the final sprint. The documentation was a product of boths contributions through joint work, and we also worked togehter on the CLI and config modules, as they are relatively small. The 3rd and 2nd sprint were also characterized by private and work-related boundaries regarding the workflow, as we both had travelled, had other due submissions, and private commitments. We managed to work around these boundaries by supporting each other and by maintaing communication about our progress and challenges.

The three sprints also helped us to improve our workflow and to find a sound balance between managing our timeand orchestrating time limitations. We also grew together, as we had to learn to adapt to each others working style and to find a common ground of understaning.

7 Sprint review

7.1 Richard Schuppener

7.2 Cem Colak

This sprint was the most chill and straightforward one, as we had already implemented all necessary functionalities and were mostly focusing on finalizing the whole software artifact and on documentation. It was rather an intentional setting, because we both have submissions in other courses until the 14. June, and we wanted to have a more relaxed sprint to be able to focus more on the other submissions. Furthermore, I learned much more about the usage of Github, how to build a CLI, what to consider while building a software artifact, how to use ML algorithms in a prediction pipeline and how to document it. With Richard I have had an excellent partner, who dedicates to deliver and also has a great sense of humor, which made the whole project much more enjoyable. Additionally, he helped me a lot with understanding specific Process Mining concepts and with the implementation of the software artifact. I'm very happy with the outcome of our work and I think we have delivered a great software artifact considering the time limitations and the complexity of the task. Finally, I'm looking foward to evaluate the software artifacta